



UNIFEI

MAXIMIZANDO A EFICIÊNCIA DE RAG: UMA ANÁLISE COMPARATIVA DE MÉTODOS RAG

**Revisão bibliográfica e plano de trabalho enviado
para análise e deliberação da banca de TCC01**

Aluno(s):

Afonso Henriques Massunari – ECO

Ryan Alves Mazzeu - ECO

Orientação:

Carlos Henrique Valério de Moraes - Orientador

Junho de 2026

Sumário

1	Introdução	2
2	Motivação	3
3	Revisão bibliográfica	4
3.1	Limitações dos LLMs e a consolidação do RAG	4
3.2	Métodos de manipulação de contexto e o compromisso de eficiência	4
3.3	Estratégias avançadas de recuperação	4
3.4	Contexto longo, estruturação documental e fragmentação	5
3.5	Infraestrutura de armazenamento e recuperação vetorial	5
3.6	Aplicações de domínio específico	5
3.7	Lacunas identificadas	5
4	Objetivos específicos	6
5	Materiais e métodos	7
6	Resultados esperados	8
7	Cronograma de atividades do TCC	9

1 Introdução

O avanço contemporâneo dos Modelos de Linguagem de Grande Porte (LLMs) expandiu significativamente as fronteiras da Inteligência Artificial, permitindo a execução de tarefas complexas que variam desde a análise de documentos legais até a geração automatizada de códigos de programação. Contudo, apesar do notável desempenho em tarefas estritamente textuais, os LLMs tradicionais enfrentam limitações severas ao lidar com conhecimentos em constante evolução e dados de domínios específicos, frequentemente resultando em respostas alucinatórias ou desatualizadas. Para mitigar esse problema, a arquitetura de Geração Aumentada de Recuperação (RAG) consolidou-se como uma abordagem não-paramétrica indispensável [1], integrando a capacidade gerativa dos modelos à precisão de bases de conhecimento externas indexadas em bancos de dados vetoriais [2]. No entanto, a implementação prática de novos sistemas RAG ainda introduz desafios complexos de engenharia e infraestrutura que exigem investigação, tais como restrições de tempo de execução, saturação de limites de tokens e o consumo elevado de recursos de hardware.

Diante desse cenário, este trabalho propõe-se a realizar uma análise investigativa das metodologias de otimização aplicadas aos processos de RAG. O escopo delineado para a pesquisa abrangerá uma avaliação comparativa detalhada entre abordagens clássicas e avançadas de recuperação — especificamente os métodos Stuff, Refine, Map Reduce, Map Re-rank, Query Step-Down e Reciprocal RAG [3]. Pretende-se analisar os impactos diretos que a escolha dessas técnicas exerce sobre três pilares fundamentais de desempenho no ecossistema de inteligência artificial: a acurácia semântica das respostas geradas (mensurada por métricas de similaridade de cosseno), a eficiência no consumo de tokens e a utilização de infraestrutura de hardware, envolvendo o consumo de CPU, memória RAM e tempo de resposta. Esse recorte metodológico justifica-se pela necessidade premente de compreender os múltiplos tradeoffs técnicos existentes entre a precisão da resposta entregue ao usuário e os custos operacionais de computação em nuvem em ambientes de alta escala.

O objetivo principal desta proposta de pesquisa é projetar e sintetizar um panorama analítico sobre a eficiência de arquiteturas RAG, buscando identificar as melhores configurações de componentes para diferentes cenários de aplicação. Especificamente, o trabalho investigará como a escolha combinada de bancos de dados vetoriais (como ChromaDB, FAISS e Pinecone), modelos de embedding (como Bge-en-small, Text-embedding-v3-large e Cohere-en-v3) e filtros de compressão contextual (limiares de similaridade e redundância) afetam o equilíbrio entre o gerenciamento de recursos e a qualidade do texto final gerado pelo modelo. Para dar sustentação teórica e metodológica a este projeto, a pesquisa tomará como base empírica e ponto de partida os dados e parâmetros discutidos na literatura recente sobre otimização sistemática por varredura em grade (grid-search), o que inclui as metodologias de testes cruzados em múltiplos domínios textuais, tais como relatórios financeiros corporativos (SEC 10Q), documentação científica (Llama2 ArXiv) e exames da área médica (MedQA) [4].

A revisão de literatura que fundamentará este projeto terá como foco publicações recentes indexadas em bases de prestígio acadêmico internacional, com ênfase em termos-chave estruturantes como Retrieval-Augmented Generation, Vector Databases, Contextual Compression e Embedding Models. Por fim, a proposta de desenvolvimento deste estudo está estruturada de forma a aprofundar essas discussões técnicas nas seções subsequentes. O plano de trabalho prevê inicialmente a apresentação do referencial teórico acerca do funcionamento matemático da similaridade de cosseno e dos algoritmos de busca por vizinhos mais próximos. Em seguida, detalha-se o cronograma e a metodologia planejada para a análise comparativa dos indicadores de desempenho de acurácia, consumo de hardware e eficácia dos filtros de compressão. Por fim, serão delineados os resultados esperados e as contribuições pretendidas para o desenvolvimento de aplicações práticas baseadas nas conclusões a serem estabelecidas pelo estudo.

2 Motivação

A justificativa para a realização deste estudo fundamenta-se nos severos desafios de engenharia e infraestrutura impostos pela operação de Modelos de Linguagem de Grande Porte (LLMs) em cenários de produção. Embora o surgimento da arquitetura de Geração Aumentada de Recuperação (RAG) tenha mitigado o problema das alucinações ao fornecer dados de fontes externas em tempo real, sua viabilidade comercial enfrenta uma barreira crítica: a ineficiência no consumo de recursos. À medida que o volume de dados organizacionais cresce, o processo tradicional de recuperar e injetar fragmentos extensos de texto diretamente nos prompts resulta em um aumento exponencial no consumo de tokens, saturação da janela de contexto e elevação dos custos com computação em nuvem. No ecossistema tecnológico atual, há uma lacuna evidente na compreensão de como as diferentes abordagens de manipulação desse contexto — como os métodos Stuff, Refine, Map Reduce, Map Re-rank, Query Step-Down e Reciprocal RAG [3] — afetam diretamente o uso de hardware, especificamente o consumo de CPU, memória RAM e tempo de resposta (latência). Investigar esses parâmetros é fundamental para a Ciência da Computação e Engenharia de Software, pois o sucesso de aplicações de inteligência artificial depende intrinsecamente da habilidade de equilibrar a acurácia da resposta gerada com a sustentabilidade financeira e computacional da infraestrutura.

A relevância deste tema acentua-se diante das tendências de mercado e dos recentes avanços científicos na área de Processamento de Linguagem Natural (PLN). Estudos recentes de otimização sistemática por varredura em grade (grid-search), que chegam a realizar mais de 23.000 iterações de testes cruzados em domínios complexos (como relatórios financeiros e dados médicos) [4], apontam que a escolha cega de componentes pode inviabilizar projetos de IA. A compressão de contexto e o uso estratégico de filtros de similaridade surgem não apenas como alternativas de refinamento, mas como requisitos obrigatórios para reduzir o tráfego desnecessário de dados. Consequentemente, a motivação para este trabalho decorre da necessidade prática de mapear esses cenários e propor um guia metodológico claro. Ao correlacionar a eficiência algorítmica com o impacto físico no hardware, este estudo conecta-se a um objetivo muito maior: democratizar e viabilizar a implementação de sistemas RAG robustos e escaláveis, reduzindo o desperdício computacional e otimizando a entrega de valor tecnológico tanto no ambiente acadêmico quanto na indústria de software.

3 Revisão bibliográfica

A literatura recente sobre Geração Aumentada de Recuperação (RAG) organiza-se em torno de um problema central: como integrar conhecimento externo a Modelos de Linguagem de Grande Porte (LLMs) de forma simultaneamente precisa e computacionalmente sustentável. Para situar a contribuição deste trabalho, esta revisão estrutura-se em cinco eixos temáticos: (i) a motivação da arquitetura RAG frente às limitações dos LLMs; (ii) os métodos de manipulação de contexto e seus compromissos de eficiência; (iii) as estratégias avançadas de recuperação; (iv) a infraestrutura de bancos de dados vetoriais; e (v) as aplicações de domínio específico. Ao fim, discutem-se as lacunas que justificam a investigação proposta.

3.1 Limitações dos LLMs e a consolidação do RAG

Os LLMs, apesar de seu desempenho notável, são propensos a alucinações e à desatualização do conhecimento paramétrico. O levantamento de Andriopoulos e Pouwelse [1] sistematiza as estratégias de aumento de conhecimento como principal caminho para prevenir alucinações, posicionando o RAG como uma alternativa não-paramétrica de baixo custo de atualização frente ao retreinamento contínuo dos modelos. Essa fragilidade também se manifesta na própria engenharia dos sistemas: Barnett et al. [5], a partir de três estudos de caso em domínios distintos (pesquisa, educação e biomedicina), catalogam sete pontos de falha recorrentes na construção de pipelines RAG, concluindo que a robustez desses sistemas é construída empiricamente durante a operação e não estabelecida em projeto. Em conjunto, esses trabalhos delimitam o problema que motiva a área, mas tratam predominantemente da acurácia e da confiabilidade, sem quantificar de forma sistemática o custo de hardware envolvido.

3.2 Métodos de manipulação de contexto e o compromisso de eficiência

O núcleo metodológico deste estudo apoia-se na caracterização dos métodos de injeção de contexto. Topsakal e Akinci [3] descrevem, em nível arquitetural, o funcionamento dos quatro fluxos fundamentais disponíveis em frameworks como o LangChain — Stuff, Map Reduce, Refine e Map Re-rank —, evidenciando trade-offs estruturais como o gargalo de paralelização do Refine e o elevado volume de chamadas do Map Reduce. Trata-se, contudo, de um trabalho descritivo, sem medições empíricas. Essa lacuna quantitativa é preenchida por Şakar e Emekci [4], que conduzem uma busca em grade de 23.625 iterações comparando seis métodos (Stuff, Refine, Map Reduce, Map Re-rank, Query Step-Down e Reciprocal RAG) e filtros de compressão contextual ao longo de múltiplos bancos vetoriais, modelos de embedding e LLMs. Os autores demonstram que não existe arquitetura universalmente ótima: enquanto o Reciprocal RAG atinge a maior similaridade, o método Stuff é 38,9% mais rápido e consome 70,5% menos tokens, e a compressão contextual reduz o uso de recursos ao custo de degradar a similaridade quando seus limiares não são calibrados por domínio. Esse trabalho constitui a principal âncora teórica e metodológica da presente pesquisa.

3.3 Estratégias avançadas de recuperação

Para além da simples injeção dos fragmentos recuperados, parte expressiva da literatura busca refinar a etapa de recuperação. Ma et al. [6] propõem o paradigma *Rewrite-Retrieve-Read*, no qual um reescritor treinado por aprendizado por reforço adapta a consulta do usuário antes da busca, mitigando a lacuna entre a pergunta e o conhecimento indexado. Em direção complementar, Rackauckas [7] combina a geração de múltiplas consultas com a fusão recíproca de postos (*reciprocal rank fusion*) no método RAG-Fusion, ampliando a abrangência das respostas, embora ao risco de desvio temático quando as consultas geradas se distan-

ciam da intenção original. Essas abordagens elevam a qualidade da recuperação, mas introduzem chamadas adicionais ao LLM, reforçando a tensão entre acurácia e custo que este trabalho pretende mensurar.

3.4 Contexto longo, estruturação documental e fragmentação

A forma como o conteúdo recuperado é apresentado ao modelo também impacta o desempenho. Liu et al. [8] demonstram empiricamente o fenômeno *lost in the middle*: o desempenho dos LLMs degrada-se quando a informação relevante se encontra no meio de contextos longos, o que questiona a estratégia de simplesmente acumular fragmentos. Como resposta, Nair et al. [9] exploram a estrutura de discurso de documentos longos para construir representações condensadas, preservando 99,6% do desempenho do melhor método zero-shot enquanto processam apenas 26% dos tokens. No nível mais elementar, Schwaber-Cohen e Patel [10] discutem estratégias de fragmentação (*chunking*) e seu efeito sobre a relevância da recuperação. Esses estudos reforçam que a eficiência em tokens — objeto deste trabalho — depende não apenas do método de injeção, mas também da estruturação prévia do contexto.

3.5 Infraestrutura de armazenamento e recuperação vetorial

O desempenho de um pipeline RAG é igualmente condicionado pela camada de armazenamento. O levantamento abrangente de Ma et al. [2] sistematiza técnicas de armazenamento e recuperação em bancos de dados vetoriais, bem como seus desafios de escalabilidade, fornecendo a base conceitual para a comparação entre soluções como ChromaDB, FAISS e Pinecone prevista nos objetivos deste estudo. A escolha desses componentes, como apontado por Şakar e Emekci [4], tem impacto direto sobre o consumo de CPU e a latência, conectando a infraestrutura de recuperação às métricas de hardware aqui investigadas.

3.6 Aplicações de domínio específico

Por fim, a literatura evidencia o potencial do RAG em domínios verticais de alta complexidade. Cui et al. [11] apresentam o Chatlaw, assistente jurídico multiagente que combina recuperação de conhecimento legal com uma arquitetura de mistura de especialistas para reduzir alucinações em um domínio sensível. No setor financeiro, Gupta [12] utiliza LLMs para analisar relatórios anuais extensos e derivar estratégias de investimento, ilustrando o desafio de processar documentos longos e ricos em jargão. Esses casos confirmam a relevância prática da otimização de RAG e dialogam com os domínios financeiro e técnico empregados como base empírica nesta pesquisa.

3.7 Lacunas identificadas

A análise conjunta revela que a literatura avançou significativamente na acurácia [6, 7], na confiabilidade [1, 5] e na compreensão das limitações de contexto [8, 9]. Entretanto, a correlação sistemática entre a escolha do método RAG e o impacto físico sobre o hardware — consumo de CPU, memória RAM e tempo de resposta — permanece pouco explorada, sendo tratada de forma agregada apenas por Şakar e Emekci [4] em escala computacional incompatível com ambientes restritos. É justamente nessa lacuna — mapear de forma reproduzível os trade-offs de eficiência de hardware entre os métodos de RAG — que este trabalho se insere.

4 Objetivos específicos

O objetivo geral desta pesquisa consiste em analisar o impacto de diferentes arquiteturas e métodos de Geração Aumentada de Recuperação (RAG) sobre a eficiência de uso de hardware, consumo de tokens e acurácia semântica, a fim de propor diretrizes técnicas para a otimização de pipelines de inteligência artificial aplicados a bases de dados complexas.

Para alcançar o propósito geral e responder à problemática de infraestrutura e custos computacionais que envolvem esses sistemas, estabelecem-se os seguintes objetivos específicos:

1. **Mapear e catalogar** as abordagens clássicas e avançadas de recuperação de contexto, com ênfase detalhada no funcionamento algorítmico dos métodos Stuff, Refine, Map Reduce, Map Re-rank, Query Step-Down e Reciprocal RAG [3].
2. **Avaliar e comparar** o desempenho de diferentes bancos de dados vetoriais (como ChromaDB, FAISS e Pinecone) em simulações cruzadas, mensurando de forma quantitativa métricas físicas de infraestrutura como o uso de CPU, consumo de memória RAM e latência de tempo de execução.
3. **Mensurar** a variação da acurácia semântica das respostas geradas a partir de métricas de similaridade de cosseno, correlacionando a precisão do texto final com a eficácia de diferentes modelos de embedding (incluindo Bge-en-small, Text-embedding-v3-large e Cohere-en-v3).
4. **Investigar** a influência e a viabilidade da aplicação de filtros de compressão contextual (focados em limites de similaridade e redundância) sobre a contenção e redução do volume total de tokens trafegados por requisição, analisando o tradeoff gerado entre a economia financeira de processamento e a eventual perda de qualidade informacional.
5. **Validar** as hipóteses e configurações propostas por meio de uma abordagem de testes sistemáticos baseada em varredura em grade (grid-search), executando os experimentos propostos sobre conjuntos de dados de múltiplos domínios públicos de alta complexidade (como relatórios SEC 10Q, Llama2 ArXiv e MedQA) [4].
6. **Sintetizar e propor** um arcabouço de boas práticas ou guia de tomada de decisão para engenheiros de software e pesquisadores, indicando as melhores combinações de componentes RAG para restrições específicas de custo, tempo ou hardware.

5 Materiais e métodos

Para a realização deste estudo comparativo, serão utilizados recursos computacionais e ferramentas de software de código aberto (open-source) amplamente consolidadas na área de inteligência artificial e processamento de linguagem natural. O ambiente de desenvolvimento será configurado em linguagem Python, utilizando frameworks como LangChain ou LlamaIndex para a orquestração e construção das pipelines de Geração Aumentada de Recuperação (RAG). Os materiais de software incluirão o uso de três bancos de dados vetoriais distintos para análise comparativa de indexação e busca: ChromaDB, FAISS e Pinecone. Para a conversão dos textos em vetores, serão integrados e avaliados três modelos de embedding: o Bge-en-small (focado em eficiência e leveza), o Text-embedding-v3-large (focado em alta dimensionalidade e precisão) e o Cohere-en-v3 (focado em desempenho robusto de classificação semântica). O hardware utilizado para a execução dos testes locais consistirá em uma estação de trabalho equipada com processador de múltiplos núcleos, memória RAM dimensionada para suportar a carga de dados vetoriais em memória e unidade de processamento gráfico (GPU) dedicada, garantindo a coleta padronizada de métricas físicas de infraestrutura.

A abordagem metodológica desta pesquisa será de natureza quantitativa e experimental, baseada em uma estratégia de otimização sistemática por varredura em grade (grid-search). O procedimento consistirá em realizar testes cruzados combinando exaustivamente cada modelo de embedding, cada banco de dados vetorial e cada um dos seis métodos de RAG definidos no escopo (Stuff, Refine, Map Reduce, Map Re-rank, Query Step-Down e Reciprocal RAG). Em cada uma dessas iterações, serão aplicados filtros de compressão contextual parametrizados com diferentes limiares de similaridade e redundância. A população de dados — ou seja, a amostra de textos que alimentará os experimentos — será composta por três conjuntos de dados públicos de múltiplos domínios e de alta complexidade estrutural, garantindo a heterogeneidade dos testes: relatórios financeiros corporativos (SEC 10Q), documentação científica (Llama2 ArXiv) e exames de múltipla escolha da área médica (MedQA).

Os instrumentos de coleta e análise de dados focarão no monitoramento em tempo real do sistema durante as consultas. O consumo de recursos de hardware será mensurado por meio de bibliotecas de monitoramento do sistema operacional em Python (como o psutil), coletando especificamente a porcentagem instantânea de uso da CPU, o pico de alocação de memória RAM e a latência total (tempo de execução em segundos) desde o envio da pergunta até a geração da resposta. Em paralelo, a eficiência de custo será contabilizada através da contagem exata de tokens de entrada (prompt) e de saída (completion) processados pelos modelos de linguagem. Por fim, a qualidade das respostas e a acurácia semântica do pipeline serão avaliadas por meio do cálculo matemático da similaridade de cosseno entre os vetores de contexto recuperados e as respostas esperadas, além da possível integração de métricas automatizadas do framework Ragas. Todos os dados coletados serão tabulados de forma estatística, permitindo a análise de correlação entre o nível de compressão do contexto e o impacto real sofrido pela infraestrutura e pela fidelidade do conteúdo gerado.

6 Resultados esperados

Os dados brutos obtidos a partir das simulações computacionais e da varredura em grade (grid-search) serão extraídos e organizados de forma puramente quantitativa e estruturada, visando garantir a total transparência e reprodutibilidade do estudo. Para facilitar a compreensão e a comparação lógica entre as variáveis, os resultados serão tabulados sequencialmente em matrizes de desempenho divididas por três eixos analíticos: eficiência de infraestrutura, eficiência de custo de tokens e acurácia semântica. Cada tabela será acompanhada de descrições estatísticas contendo medidas de resumo adequadas, tais como a média aritmética e o desvio padrão dos tempos de execução e do uso de memória RAM, além do cálculo de frequências absolutas e relativas para identificar quais combinações de modelos de embedding, bancos vetoriais e métodos de RAG apresentaram os comportamentos mais estáveis.

No eixo de eficiência de infraestrutura, os resultados principais detalharão o impacto físico de cada método de RAG (Stuff, Refine, Map Reduce, Map Re-rank, Query Step-Down e Reciprocal RAG) sobre o hardware. Os dados de consumo de CPU (mensurados em porcentagem de uso) e de memória RAM (mensurados em megabytes de pico de alocação) serão dispostos em colunas comparativas associadas a cada banco de dados vetorial testado (ChromaDB, FAISS e Pinecone). A latência temporal de cada consulta será documentada em segundos de forma isolada, permitindo correlacionar diretamente o tempo de resposta com a complexidade do algoritmo de recuperação empregado. Gráficos de barras agrupadas serão gerados para ilustrar o comportamento do hardware em relação ao tamanho e profundidade das janelas de contexto enviadas ao modelo gerativo.

No que tange aos custos e à qualidade informacional, a seção apresentará de maneira objetiva a contagem de tokens trafegados nas APIs dos LLMs, discriminando os valores exatos de tokens de entrada e de saída para cada iteração do experimento. Em paralelo, os coeficientes numéricos obtidos através do cálculo da similaridade de cosseno e das métricas do framework Ragas serão organizados em tabelas de dupla entrada. Essas matrizes cruzarão os diferentes limiares de filtros de compressão contextual aplicados aos múltiplos domínios (SEC 10Q, Llama2 ArXiv e MedQA) com as pontuações de relevância alcançadas por cada modelo de embedding (Bge-en-small, Text-embedding-v3-large e Cohere-en-v3). A exposição desses indicadores puramente matemáticos e estatísticos fornecerá uma base factual sólida, livre de interpretações subjetivas, mapeando detalhadamente os tradeoffs técnicos existentes na arquitetura do sistema.

7 Cronograma de atividades do TCC

As atividades planejadas para a execução deste Trabalho de Conclusão de Curso (TCC) estão distribuídas ao longo de cinco fases principais (F.1 a F.5), detalhadas na Tabela 1. A distribuição temporal destas atividades entre os meses de maio e setembro de 2026 está ilustrada nos gráficos de Gantt na sequência.

F.1: Proposição e Delimitação
Definição do tema, objetivos, pergunta de pesquisa e escopo do trabalho.
Atividades planejadas para a fase 1
Elaboração do anteprojeto; reunião com orientador; definição da metodologia.
Resultados esperados para a fase 1
Anteprojeto aprovado pelo orientador e plano de trabalho definido.
F.2: Revisão Bibliográfica
Levantamento e síntese da literatura relevante para fundamentar o TCC.
Atividades planejadas para a fase 2
Pesquisa em bases; leitura crítica; organização das referências (BibTeX).
Resultados esperados para a fase 2
Estado da arte consolidado e seção de revisão escrita.
F.3: Metodologia e Implementação
Desenvolvimento do método, ferramentas e implementação do experimento/protótipo.
Atividades planejadas para a fase 3
Desenho experimental; implementação do protótipo; testes iniciais.
Resultados esperados para a fase 3
Protótipo funcional e conjunto de dados/resultados preliminares.
F.4: Análise, Redação e Entrega
Análise dos resultados, redação dos capítulos e preparação da versão final.
Atividades planejadas para a fase 4
Análise estatística/qualitativa; escrita dos capítulos; revisão com orientador.
Resultados esperados para a fase 4
Versão final do relatório entregue e pronta para defesa.
F.5: Preparação da Apresentação/Defesa
Preparação de slides, ensaio e ajustes finais para a defesa.
Atividades planejadas para a fase 5
Criação de apresentação; ensaios; correções finais.
Resultados esperados para a fase 5
Apresentação pronta e cronograma de defesa agendado.

Tabela 1: Detalhamento das fases do TCC

Distribuição Temporal (2026)

1º Semestre					
Jan	Fev	Mar	Abr	Mai	Jun
				F.1	
				F.2	
					F.3

2º Semestre					
Jul	Ago	Set	Out	Nov	Dez
F.3					
	F.4				
		F.5			

Referências Bibliográficas

- [1] K. Andriopoulos and J. Pouwelse, “Augmenting LLMs with knowledge: A survey on hallucination prevention,” arXiv preprint arXiv:2309.16459, 2023.
- [2] L. Ma, R. Zhang, Y. Han, S. Yu, Z. Wang, Z. Ning, J. Zhang, P. Xu, P. Li, Z. Qiao, W. Ju, C. Chen, D. Wang, K. Liu, P. Wang, P. Wang, Y. Fu, C. Liu, Y. Zhou, and C.-T. Lu, “A comprehensive survey on vector database: Storage and retrieval technique, challenge,” arXiv preprint arXiv:2310.11703, 2024.
- [3] O. Topsakal and T. C. Akinci, “Creating large language model applications utilizing LangChain: A primer on developing LLM apps fast,” in *Proceedings of the 5th International Conference on Applied Engineering and Natural Sciences (ICAENS)*, Konya, Turkey, 2023.
- [4] T. Şakar and H. Emekci, “Maximizing RAG efficiency: A comparative analysis of RAG methods,” *Natural Language Processing*, vol. 31, pp. 1–25, 2025.
- [5] S. Barnett, S. Kurniawan, S. Thudumu, Z. Brannelly, and M. Abdelrazek, “Seven failure points when engineering a retrieval augmented generation system,” arXiv preprint arXiv:2401.05856, 2024.
- [6] X. Ma, Y. Gong, P. He, H. Zhao, and N. Duan, “Query rewriting for retrieval-augmented large language models,” arXiv preprint arXiv:2305.14283, 2023.
- [7] Z. Rackauckas, “RAG-Fusion: A new take on retrieval-augmented generation,” arXiv preprint arXiv:2402.03367, 2024.
- [8] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, “Lost in the middle: How language models use long contexts,” arXiv preprint arXiv:2307.03172, 2023.
- [9] I. Nair, S. Somasundaram, A. Saxena, and K. Goswami, “Drilling down into the discourse structure with LLMs for long document question answering,” arXiv preprint arXiv:2311.13565, 2023.
- [10] R. Schwaber-Cohen and A. Patel, “Chunking strategies for LLM applications,” Pinecone Learning Center, 2025, disponível em: <https://www.pinecone.io/learn/chunking-strategies/>.
- [11] J. Cui, M. Ning, Z. Li, H. Li, Y. Yan, B. Chen, B. Ling, Y. Tian, and L. Yuan, “Chatlaw: A multi-agent legal assistant based on a role-aligned mixture-of-experts architecture,” arXiv preprint arXiv:2306.16092, 2023.
- [12] U. Gupta, “GPT-InvestAR: Enhancing stock investment strategies through annual report analysis with large language models,” arXiv preprint arXiv:2309.03079, 2023.